

Лабораторная работа №5

Отчёт

Коровкин Никита Михайлович

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	10
	Список литературы	11

Список иллюстраций

2.1	Создание файла	6
2.2	Код	6
2.3	Тестирование программы	7
2.4	Код новой программы	7
2.5	Запуск программы	7
2.6	Код	8
2.7	Компиляция и попытка открытия исходника	8
2.8	Операции без Stickybit	9

Список таблиц

1 Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

2 Выполнение лабораторной работы

Создадим файл с кодом (рис. ??).

```
nmkorovkin@username:~$ touch simpleid.c
nmkorovkin@username:~$ nano simpleid.c
```

Рис. 2.1: Создание файла

И запишем туда следующий код (рис. 2.2).

```
GNU nano 8.1 simpleid.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t uid = geteuid ();
    gid_t gid = getegid ();
    printf ("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

Рис. 2.2: Код

Скомпилируем программу, запустим её. Как видим, она практически идентична команде `id` (рис. 2.3).

```
nmkorovkin@username:~$ gcc simpleid.c -o simpleid
nmkorovkin@username:~$ ./simpleid
uid=1000, gid=1000
nmkorovkin@username:~$ id
uid=1000(nmkorovkin) gid=1000(nmkorovkin) groups=1000(nmkorovkin),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
nmkorovkin@username:~$
```

Рис. 2.3: Тестирование программы

Создадим новую программу (рис. 2.4).

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid,
    ,→ real_gid);
    return 0;
}
```

Рис. 2.4: Код новой программы

также скомпилируем её и запустим (рис. 2.5).

```
root@username:~# chown root:guest /home/guest/simpleid2
chmod u+s /home/guest/simpleid2
chown: cannot access '/home/guest/simpleid2': No such file or directory
chmod: cannot access '/home/guest/simpleid2': No such file or directory
```

Рис. 2.5: Запуск программы

Создадим новый файл, и напишем туда код (рис. 2.6).

```

#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}

```

Рис. 2.6: Код

Скомпилируем программу и попробуем открыть исходный код (рис. 2.7).

Компиляция и попытка открытия исходника

Рис. 2.7: Компиляция и попытка открытия исходника

Теперь сменим для исполняемого файла владельца

Попробуем с помощью программы прочитать исходный код, и это успешно получается

Прочтем файл /etc/shadow

Теперь посмотрим, есть ли на папке /tmp stickybit. После этого создадим от лица пользователя guest файл.

От лица другого пользователя пытаемся дозаписать что-то в файл, записать что-то в файл и удалить этот файл. Все операции неуспешны

Снимем Stickybit и попробуем повторить эти операции. Теперь получилось удалить файл (рис. 2.8).

Операции без Stickybit

Рис. 2.8: Операции без Stickybit

3 Выводы

В результате выполнения лабораторной работы были получены знания механизмов изменения идентификаторов, применения SetUID- и Sticky-битов

Список литературы